

1. Obiettivo e Scelta Tecnologica

In conformità con le specifiche richieste, il gruppo ha optato per il **Caso A (Ristrutturazione del 1° Progetto)**. L'applicativo originale, originariamente sviluppato in PHP procedurale, è stato interamente riprogettato utilizzando la tecnologia **Java (Jakarta EE per la versione java21/tomcat11 - javax per la versione java8/tomcat9)**, eseguito su server **Apache Tomcat**, con l'integrazione del motore di template **Thymeleaf** per il rendering lato server.

2. Architettura Software (Pattern MVC)

Il sistema è stato rifattorizzato seguendo rigorosamente il pattern architetturale **Model-View-Controller (MVC)** per garantire un'alta coesione e un basso accoppiamento:

- **Model:** Rappresentato da classi Java (POJO) che mappano le entità del dominio (Cliente, Contratto, Ombrellone).
- **Controller:** Implementato tramite classi Servlet, mappate con l'annotazione `@WebServlet`. I Controller si occupano esclusivamente di validare l'input HTTP, orchestrare la logica di business e smistare la risposta alla View corretta o generare un payload JSON.
- **View:** Costruita con pagine HTML arricchite dai tag Thymeleaf. La scelta di Thymeleaf ha permesso di adottare un approccio a frammenti, centralizzando elementi UI ricorrenti come l'header, la sidebar, i fogli di stile e i modali, eliminando completamente la duplicazione del codice frontend.

3. Gestione dei Dati (Pattern DAO e Sicurezza)

L'accesso al database MySQL è stato isolato tramite il Data Access Object Pattern. Questa scelta progettuale ha portato due grandi vantaggi:

1. **Sicurezza:** Tutte le query SQL utilizzano la classe `PreparedStatement`. Questo garantisce l'igienizzazione automatica dei parametri di input, mitigando radicalmente le vulnerabilità da *SQL Injection*.
2. **Integrità Transazionale:** Nelle operazioni complesse, come la registrazione di una nuova prenotazione (che coinvolge la creazione di un cliente, del contratto e l'occupazione giornaliera dell'ombrellone), la gestione del database è stata incapsulata in **transazioni manuali** disabilitando l'autocommit (`conn.setAutoCommit(false)`). Questo garantisce le proprietà ACID: in caso di sovrapposizione di date o errori, viene effettuato un `rollback` completo, prevenendo inconsistenze a database.

4. Interazione Frontend-Backend (API Asincrone)

Per modernizzare la User Experience e ridurre i tempi di caricamento, molte funzionalità (come il calcolo dinamico delle tariffe, il controllo di disponibilità e la gestione asincrona dei modali CRUD) non ricaricano l'intera pagina.

Il frontend interroga le Servlet Java tramite chiamate asincrone JavaScript. Per permettere alle Servlet di interpretare correttamente i payload di tipo `multipart/form-data` inviati dal frontend tramite l'oggetto `FormData`, i Controller sono stati decorati con l'annotazione Jakarta `@MultipartConfig`. Le risposte vengono poi restituite e parsate in formato JSON.